Protection Needs Elicitation (PNE) & CIA Mapping

COMSATS University Islamabad | CYC386 Secure Software Design and Development
Team: Talal Nadeem Awan (FA23-BCT-051) & Shahroze Afaq (FA23-BCT-039)

Protection Needs Elicitation (PNE) & CIA Mapping Report

Course: CYC386 – Secure Software Design and Development

Exam: Midterm Lab Exam — 1-Week DevSecOps Security Sprint (Fall 2026)

Institution: COMSATS University Islamabad, Department of Computer Science

Instructor: Engr. Muhammad Ahmad Nawaz

Project: SecureDrop – Enterprise Document Vault Platform (`SecureApp-Sprint-Team1`)

Team Members

- **Talal Nadeem Awan** (FA23-BCT-051)
- **Shahroze Afaq** (FA23-BCT-039)

---

1. Executive Overview

In accordance with CDF Unit 2, Protection Needs Elicitation (PNE) forms the foundational activity of the Secure Software Development Life Cycle (SSDLC). This report identifies critical system assets, elicits protection needs across multiple stakeholder perspectives, maps every security objective to the **CIA Triad** (Confidentiality, Integrity, Availability), and derives quantifiable, testable security requirements.

---

2. Comprehensive Asset Inventory

Assets represent items of value that require protection against unauthorized disclosure, alteration, destruction, or interruption.

Asset ID	Asset Name	Asset Type	Description	Sensitivity / Criticality
AST-01	User Credentials & Passwords	Data Asset	Password hashes stored in SQLite database (`users` table).	**Critical** (Confidentiality & Integrity)
AST-02	User Session Tokens & Cookies	Data Asset	HTTP session state cookies and cryptographic session secrets.	**Critical** (Confidentiality & Integrity)
AST-03	Anti-CSRF Synchronizer Tokens	Data Asset	Per-session cryptographically random tokens validating state mutations.	**High** (Integrity)
AST-04	Uploaded User Documents	Data Asset	Proprietary text records and documents stored on disk (`uploads/`).	**Critical** (Confidentiality & Integrity)

| **AST-05** | Database File & Schema | Data / Storage | `secureapp.db` holding relational metadata, user identities, and ownership maps. | **Critical** (CIA Triad) |

| **AST-06** | File Processing Worker (`processor.py`) | Process Asset | Background Python sub-process parsing document content and verifying integrity. | **High** (Integrity & Availability) |

| **AST-07** | Native Metadata Parser (`metadata_parser.exe`)| Process / Binary | Compiled C binary performing native header and length validation. | **High** (Integrity & Availability) |

| **AST-08** | Operating System Shell & Kernel Space | Infrastructure | Underlying host execution environment executing Python and binary processes. | **Critical** (Confidentiality, Integrity, Availability) |

| **AST-09** | CI/CD Build Infrastructure | Pipeline Asset | GitHub Actions runner automating build, SAST, and automated testing gates. | **High** (Integrity) |

---

3. Stakeholder Protection Needs Elicitation

3.1 End User Perspective

- **PN-USR-01:** Users require that their private uploaded files remain strictly inaccessible to any other user or unauthorized third party.
- **PN-USR-02:** Users require guarantee that their documents cannot be modified or deleted by another party without their explicit consent.
- **PN-USR-03:** Users require that their authentication sessions cannot be forged or hijacked via cross-site requests or hidden overlay clicks.
- **PN-USR-04:** Users require constant, uninterrupted availability to access and download their stored files.

3.2 System Administrator Perspective

- **PN-ADM-01:** Administrators require that user passwords are stored using salted, one-way cryptographic hashing algorithms (`werkzeug.security` pbkdf2/scrypt).
- **PN-ADM-02:** Administrators require isolation between user files and system files, preventing path traversal or execution of uploaded binaries.
- **PN-ADM-03:** Administrators require that web input cannot compromise the operating system through arbitrary shell execution.

3.3 Security / DevOps Operator Perspective

- **PN-OPS-01:** Operators require that the native C helper execution cannot overwrite stack memory or crash the application service (Denial of Service).
- **PN-OPS-02:** Operators require automated security gates in CI/CD that fail builds whenever High or Critical vulnerabilities are introduced.
- **PN-OPS-03:** Operators require auditability of code changes via pull request peer reviews and branch protection rules.

3.4 Attacker Threat Model (Adversarial Perspective)

- **PN-ATK-01:** Adversaries will attempt to manipulate resource identifiers (File IDs) to access other users' sensitive documents (IDOR).
- **PN-ATK-02:** Adversaries will attempt to inject shell operators (`;`, `&`, `|`, `$()`) into filename inputs to obtain Remote Code Execution (OS Command Injection).

- **PN-ATK-03:** Adversaries will craft oversized metadata payloads exceeding fixed buffers to corrupt stack memory or hijack control flow (Buffer Overflow).
- **PN-ATK-04:** Adversaries will host malicious sites sending unauthorized POST requests to delete or alter user data (CSRF).
- **PN-ATK-05:** Adversaries will frame the application inside transparent iframes to trick users into performing unintended actions (Clickjacking).

---

4. Explicit CIA Triad Mapping Matrix

Every protection need elicited must be systematically mapped to one or more pillars of the CIA Triad:

| Need ID | Protection Need Description | Confidentiality | Integrity | Availability | Mapped Vulnerability Class / Threat |

| :--- | :--- | :---: | :---: | :---: | :--- |

| **PN-01** | Document access restriction to legitimate owners only | **X** | | | CWE-639: Insecure Direct Object Reference (IDOR) |

| **PN-02** | Prevention of unauthorized document deletion / modification | | **X** | **X** | CWE-639 (IDOR) & CWE-352 (CSRF) |

| **PN-03** | Elimination of unauthorized shell command execution | **X** | **X** | **X** | CWE-78: OS Command Injection |

| **PN-04** | Prevention of memory corruption and service crashes in native parsers | **X** | **X** | **X** | CWE-120 / CWE-121: Buffer Overflow |

| **PN-05** | Prevention of cross-origin state-changing forged requests | | **X** | **X** | CWE-352: Cross-Site Request Forgery (CSRF) |

| **PN-06** | Mitigation of UI redressing and transparent iframe framing | **X** | **X** | | CWE-1021: Clickjacking / UI Redressing |

| **PN-07** | Protection of stored user credentials against offline cracking | **X** | **X** | | CWE-259 / CWE-307: Credential Exposure |

| **PN-08** | Session fixation and cookie theft prevention | **X** | **X** | | Session Cookie Security (`HttpOnly`, `SameSite=Strict`) |

| **PN-09** | Restriction of uploaded file sizes to prevent storage exhaustion | | | **X** | Denial of Service (`MAX_CONTENT_LENGTH = 5MB`) |

| **PN-10** | Automated failure of CI pipeline on High/Critical vulnerabilities | | | **X** | | DevSecOps Pipeline Security Gate (SAST/DAST) |

---

5. Derived Measurable Security Requirements

From the CIA mapping above, the engineering team establishes the following measurable, testable security requirements:

1. **SR-01 (Object-Level Authorization - IDOR):**

   Requirement: Every database retrieval, processing invocation, download, and deletion query MUST enforce ownership validation (`WHERE id = ? AND owner_id = session['user_id']`). Any attempt to access a resource owned by another user MUST return HTTP `404 Not Found` (or `403 Forbidden`).

Verification: `test_03_idor_before_after` executes cross-user download attempt and confirms zero information leakage.

2. **SR-02 (Shell Metacharacter Elimination - Command Injection):**

Requirement: No user-controlled string shall ever be concatenated into a shell interpreter string. All external process invocations MUST utilize `subprocess.run([...], shell=False)` with argument arrays and strict regex validation (`^[A-Za-z0-9_-]{1,64}\.txt$`).

Verification: `test_02_command_injection_before_after` submits `report.txt & echo LAB_MARKER` and confirms rejection with non-zero exit code.

3. **SR-03 (Native Memory Safety & Bounds Checking - Buffer Overflow):**

Requirement: All native C modules MUST validate input string lengths prior to memory copies. Buffers MUST have fixed maximum sizes, verify `len < sizeof(buffer)`, use bounded copy routines (`memcpy`), and be compiled with compiler stack canaries (`-fstack-protector-strong`) and FORTIFY_SOURCE (`-D_FORTIFY_SOURCE=2`).

Verification: `test_01_buffer_overflow_before_after` passes a 63-byte input into a 32-byte buffer and verifies safe rejection without stack corruption.

4. **SR-04 (Anti-CSRF Synchronizer Tokens):**

Requirement: All state-mutating HTTP requests (`POST`, `PUT`, `DELETE`) MUST require a cryptographically secure 256-bit synchronizer token (`session['csrf_token']`) validated with constant-time comparison (`secrets.compare_digest`). Additionally, session cookies MUST specify `SameSite=Strict` and `HttpOnly=True`.

Verification: `test_04_csrf_before_after` submits forged state-changing deletion request without token and verifies HTTP 403 response.

5. **SR-05 (Anti-Clickjacking Framing Defenses):**

Requirement: All HTTP responses served by the application MUST include `X-Frame-Options: DENY` and `Content-Security-Policy: default-src 'self'; frame-ancestors 'none';`.

Verification: `test_05_clickjacking_headers_before_after` inspects HTTP response headers and verifies browser framing blockage.

6. **SR-06 (Automated DevSecOps Quality Gate):**

Requirement: The GitHub Actions CI/CD pipeline MUST execute automated SAST scanning (Bandit) on every commit and pull request. The build step MUST exit with non-zero status and fail the pipeline whenever High or Critical severity vulnerabilities are detected.

Verification: Pipeline logs show exit code 1 on vulnerable branches and exit code 0 on remediated branches.

Report Compiled & Verified by: Talal Nadeem Awan (FA23-BCT-051) & Shahroze Afaq (FA23-BCT-039)